

Laboratorio di Calcolo

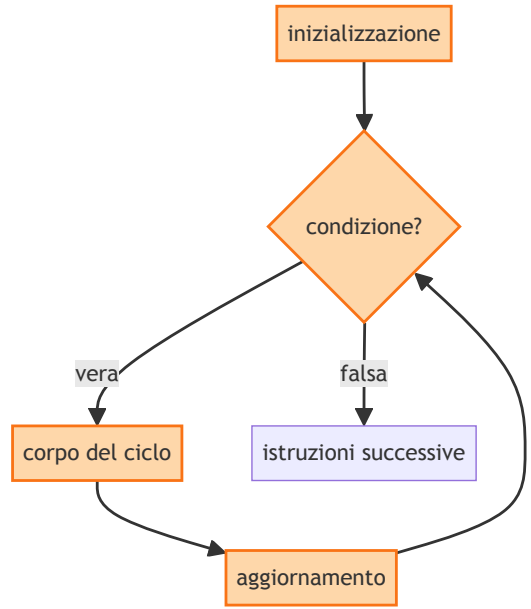
A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Iterazioni

Ripetere le istruzioni

- Nella lezione 12 abbiamo stampato una tabella scrivendo una `printf` **per ogni riga**: con 1000 righe?
- Gran parte del calcolo scientifico consiste nel **ripetere** la stessa operazione: sommare i termini di una serie, avanzare il tempo di un passo, leggere N misure…
- L'**iterazione** (o *ciclo*, *loop*) è la terza struttura della programmazione strutturata (lezione 14).
- In C ci sono tre cicli: `while`, `do ... while` e `for`.



Ogni ciclo ha quattro ingredienti: **inizializzazione**, **condizione** di permanenza, **corpo** e **aggiornamento**.

Il ciclo while

`while (condizione) { corpo }` : finché la condizione è vera, esegui quanto contenuto nel corpo

```c
#include <stdio.h>

int main() {
    const double T12 = 5730.0;   // emivita del C-14 [anni]
    double A = 1000.0;           // attivita' [Bq]
    double t = 0.0;              // [anni]
    int n = 0;                   // tempi di dimezzamento

    while (A > 1.0) {
        A = A / 2;
        t = t + T12;
        n++;
    }
    printf("n = %d, t = %g anni, A = %g Bq\n", n, t, A);
    return 0;
}
```

```
$ ./decadimento
n = 10, t = 57300 anni, A = 0.976562 Bq
```

- Domanda: dopo quanto tempo l'attività scende sotto 1 Bq?
- La condizione è controllata **prima** di ogni ripetizione: se è falsa da subito, il corpo non viene mai eseguito.
- Il corpo deve **modificare** qualcosa da cui dipende la condizione, altrimenti il ciclo non finisce più.

Contatori e accumulatori

Contatore

Conta quante volte succede qualcosa.

```
1   int n = 0;          // inizializzazione!
2   ...
3   n++;                // nel corpo del ciclo
```

Accumulatore

Accumula somme (o prodotti) di valori.

```
1   double somma = 0.0;     // inizializzazione della somma
2   double prod  = 1.0;     // inizializzazione del prodotto
3   ...
4   somma += x;
5   prod  *= x;
```

- Contatori e accumulatori vanno **inizializzati prima** del ciclo.
- Se dimenticate l'inizializzazione, partono da un valore **a caso**. `-Wall` spesso se ne accorge:

```
1   warning: ‘somma’ is used uninitialized
2     [-Wuninitialized]
```

- La **condizione di arresto** deve prima o poi diventare vera: un contatore che cresce, una quantità che diminuisce, una precisione raggiunta.

Il ciclo for

Quando il numero di ripetizioni è noto, i quattro ingredienti stanno **su una riga**.

```
for (inizializzazione; condizione; aggiornamento) {
    corpo;
}
```

è equivalente a

```
inizializzazione;
while (condizione) {
    corpo;
    aggiornamento;
}
```

Il contatore si può dichiarare **nel** `for`: `for (int i = 0; i < 5; i++)`. Così `i` esiste **solo dentro il ciclo**: nell'esempio qui accanto non potremmo stamparla dopo.

```
int i;
for (i = 0; i < 5; i++) {
    printf("%d ", i);
}
printf("\nfine: i = %d\n", i);
```

```
0 1 2 3 4
fine: i = 5
```

- `i < 5` partendo da `0`: esattamente **5** ripetizioni. È la forma tipica in C.
- L'aggiornamento avviene **dopo** il corpo: all'uscita `i` vale `5`.
- Si può anche contare all'indietro (`i--`) o a passi diversi (`i += 2`).

Esempio: tabella di valori

Il moto uniformemente accelerato delle lezioni 11 e 12, finalmente senza una `printf` per riga.

```c
#include <stdio.h>

int main() {
    const double a = 2.0;    // [m/s^2]
    double x0 = 1.0;         // [m]
    double v0 = 3.0;         // [m/s]
    double dt = 0.5;         // [s]

    printf("# t[s]      x[m]\n");
    // Se i non mi serve dopo il ciclo:
    for (int i = 0; i <= 6; i++) {
        double t = i * dt;
        double x = x0 + v0 * t + 0.5 * a * t * t;
        printf("%6.2f %9.4f\n", t, x);
    }
    return 0;
}
```

```
$ ./tabella
# t[s]      x[m]
  0.00    1.0000
  0.50    2.7500
  1.00    5.0000
  1.50    7.7500
  2.00   11.0000
  2.50   14.7500
  3.00   19.0000
```

- Per 1000 righe basta cambiare `6` in `1000`.

Esempio: la serie di Basilea

Eulero (1734): $\sum_{k=1}^{\infty} \frac{1}{k^2} = \frac{\pi^2}{6}$. Quanti termini servono?

```c
#include <stdio.h>
#include <math.h>

int main() {
    int N;
    double somma = 0.0;
    printf("Numero di termini: ");
    scanf("%d", &N);

    for (int k = 1; k <= N; k++) {
        somma += 1.0 / ((double) k * k);
    }

    double esatto = M_PI * M_PI / 6;
    printf("Somma:   %.10f\n", somma);
    printf("pi^2/6:  %.10f\n", esatto);
    printf("Errore:  %.3e\n", esatto - somma);
    return 0;
}
```

`N`	Somma	Errore
10	`1.5497677312`	`9.517e-02`
1000	`1.6439345667`	`9.995e-04`
1000000	`1.6449330668`	`1.000e-06`

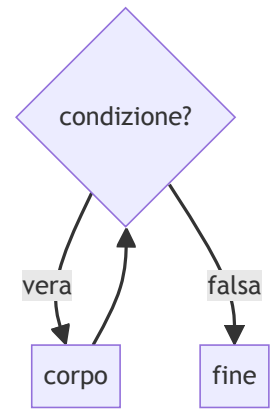
- Errore $\approx 1/N$: ogni cifra in più costa 10 volte più termini.
- `(double) k * k`: tra `int`, `k * k` va in **overflow** per $k > 46340$ (lezione 11).
- `1.0 / ...`: con `1 / ...` la divisione intera darebbe `0` (lezione 10).

Il ciclo do … while

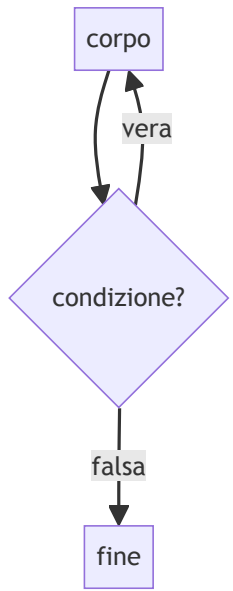
```
1   do {
2       corpo;
3   } while (condizione);    // notate il ;
```

- La condizione è controllata **dopo** il corpo: il corpo viene eseguito **almeno una volta**.
- Utile quando la condizione dipende da qualcosa che si calcola **dentro** il ciclo.
- Esempio tipico: un metodo iterativo che si ferma quando due stime successive sono abbastanza vicine.

while



do … while



Erone: dallo pseudocodice al C

Nella lezione 7 avevamo scritto l'algoritmo in pseudocodice: ora ogni passo diventa un'istruzione C.

Pseudocodice (lezione 7)

```
leggi a, ε
x ← a
ripeti:
    x_nuovo ← (x + a/x) / 2
    se |x_nuovo - x| < ε: esci
    x ← x_nuovo
stampa x_nuovo
```

"ripeti … esci" è un `do … while` : il controllo sta **in fondo**. Per questo salviamo la differenza in `diff` **prima** di aggiornare `x` .

C

```c
#include <stdio.h>
#include <math.h>

int main() {
    const double EPS = 1e-12;
    double a, x, x_nuovo, diff;

    scanf("%lf", &a);
    x = a;
    do {
        x_nuovo = 0.5 * (x + a / x);
        diff = fabs(x_nuovo - x);
        x = x_nuovo;
    } while (diff >= EPS);
    printf("%.12f\n", x_nuovo);
    return 0;
}
```

Esempio: media di N misure

```c
#include <stdio.h>
#include <math.h>

int main() {
    int N;
    double x, somma = 0.0, somma2 = 0.0;

    printf("Numero di misure: ");
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        printf("Misura %d [s]: ", i + 1);
        scanf("%lf", &x);
        somma  += x;
        somma2 += x * x;
    }

    double media = somma / N;
    double sigma = sqrt((somma2 - N * media * media) / (N - 1));
    printf("\nMedia = %.4f s, sigma = %.4f s\n", media, sigma);
    return 0;
}
```

```
$ ./media
Numero di misure: 5
Misura 1 [s]: 2.01
Misura 2 [s]: 1.98
Misura 3 [s]: 2.03
Misura 4 [s]: 2.00
Misura 5 [s]: 1.97

Media = 1.9980 s, sigma = 0.0239 s
```

- $\sigma = \sqrt{\frac{\sum x_i^2 - N\bar{x}^2}{N-1}}$: bastano **due accumulatori**, senza conservare le misure.
- Attenzione: con misure molto precise questa formula soffre di **cancellazione** (lezione 4).

Cicli infiniti

```c
int i = 0;
while (i < 10) {
    printf("%d\n", i);
    // manca i++ !
}
```

```
$ ./infinito
0
0
0
...
^C
```

- La condizione non diventa **mai** falsa: il programma non termina.
- **Ctrl+C** interrompe il programma in esecuzione nel terminale.

- Cause tipiche:
  - aggiornamento dimenticato;
  - aggiornamento nella direzione sbagliata (`i--` invece di `i++`);
  - condizione che non può diventare falsa (`x != 1.0` con `x` reale);
  - metodo iterativo che non converge.
- Se un ciclo **può** non convergere, aggiungete un limite al numero di iterazioni:

```c
} while (fabs(x_nuovo - x) > EPS && n < N_MAX);
```

Le trappole dei cicli

1. Il punto e virgola dopo for

```c
for (i = 0; i < 5; i++);
{
    printf("i = %d\n", i);
}
```

```
i = 5
```

- Il corpo del `for` è l'istruzione vuota `;`: il ciclo gira 5 volte senza fare nulla.
- Il blocco viene eseguito **una volta**, dopo il ciclo.
- È codice valido: non sempre il compilatore vi avvisa.

2. Errore di uno (*off-by-one*)

Ciclo	Ripetizioni
`for (i = 0; i < 10; i++)`	10
`for (i = 1; i < 10; i++)`	**9**
`for (i = 0; i <= 10; i++)`	**11**
`for (i = 1; i <= 10; i++)`	10

- Partendo da `k = 0` nella serie di Basilea, il primo termine è `1.0 / 0`: la somma diventa `inf`!
- Controllate **primo** e **ultimo** valore del contatore: è il test più semplice.

Contatore double o int?

```c
double t;
for (t = 0.0; t <= 0.3; t += 0.1) {
    printf("%.2f\n", t);
}
```

```
0.00
0.10
0.20
```

- Manca `0.30` ! Dopo tre somme `t` vale `0.30000000000000004` , che è **maggiore** di `0.3` .
- Sommare 10 volte `0.1` dà `0.99999999999999989` , non `1` (lezione 4): con `t != 1.0` il ciclo non finirebbe mai.

```c
int i;
for (i = 0; i <= 3; i++) {
    t = i * 0.1;
    printf("%.2f\n", t);
}
```

```
0.00
0.10
0.20
0.30
```

Contate con un **intero**, poi calcolate `t = i * dt` : il numero di passi è esatto e gli errori non si **accumulano**.

Quale ciclo scegliere?

Ciclo	Quando usarlo	Esempio
`for`	Numero di ripetizioni **noto** in anticipo	Tabella di N valori, somma di N termini, N misure
`while`	Si ripete finché vale una condizione, che può essere falsa **da subito**	Dimezzamenti fino a una soglia, leggere fino alla fine dei dati
`do ... while`	Il corpo va eseguito **almeno una volta**, la condizione dipende dal corpo	Metodi iterativi (Erone)

- I tre cicli sono **equivalenti**: ognuno si può riscrivere con gli altri. Scegliete quello che rende il codice più **leggibile**.
- Prima di scrivere un ciclo chiedetevi: da dove parte? Cosa cambia a ogni passo? **Quando si ferma?**

Controlliamo se abbiamo capito…

Cosa stampa ogni `printf` ?

```c
int i, n = 0, s = 0;
for (i = 0; i < 10; i += 3) n++;
printf("%d\n", n);
i = 10;
while (i < 5) i++;
printf("%d\n", i);
i = 10;
do { i++; } while (i < 5);
printf("%d\n", i);
for (i = 5; i > 0; i--) s += i;
printf("%d\n", s);
for (i = 1; i <= 100; i *= 2) { }
printf("%d\n", i);
```

Output	Perché
`4`	`i` vale 0, 3, 6, 9
`10`	La condizione è falsa subito: zero ripetizioni
`11`	Il `do` esegue il corpo almeno una volta
`15`	5 + 4 + 3 + 2 + 1
`128`	1, 2, 4, …, 64 → 128 > 100